

Basketball Shot Game — Unity Build Spec

Oct 7, 2026 · @KK

Build a mobile basketball game in Unity for Android and iOS. A player picks up a ball, aims by touch and shoots at the hoop. While aiming, the screen shows the arc the ball will take and the physics behind it.

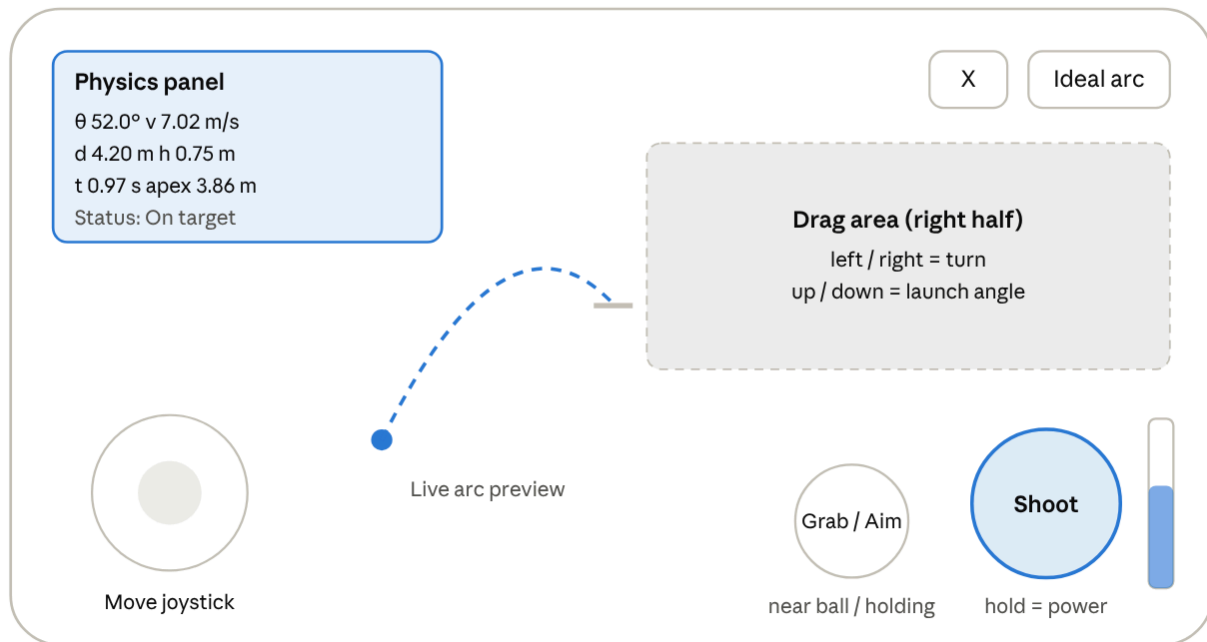
What is provided

File	Contents	Note
Basketball Ground.fbx	Court and hoop	Static scene
Basketball.fbx	Ball	Needs a Rigidbody and SphereCollider
Player01.fbx	Player character	Rigged with a skeleton, no animations

What is required

1. **Scene at real scale:** 1 unit = 1 m, rim 3.05 m high, ball 0.24 m across. Colliders on the floor, backboard and rim.
2. **Animations:** set Player01 up as Humanoid and source the Idle, Walk, Pick Up, Aim and Shoot clips (for example from Mixamo).
3. **Movement:** a virtual joystick, bottom-left.
4. **Pick up:** a Grab button appears near the ball; the ball then attaches to the right hand.
5. **Aiming by the player:** drag to turn the player and set the launch angle (25°–75°). Hold Shoot to charge power, from 4 to 14 m/s. Release to throw.
6. **Arc preview:** while aiming, draw the full predicted path and update it live. Green means it goes in, white means a miss.
7. **Physics panel:** show the angle, speed, distance, height, flight time, apex height, ideal speed, and "short / long by X m".
8. **Real flight:** the ball is thrown with the computed velocity under Unity gravity (9.81 m/s², no drag) and must follow the drawn arc.
9. **Scoring and respawn** after each shot.

Left thumb moves, right thumb aims and shoots



mobile screen layout · landscape

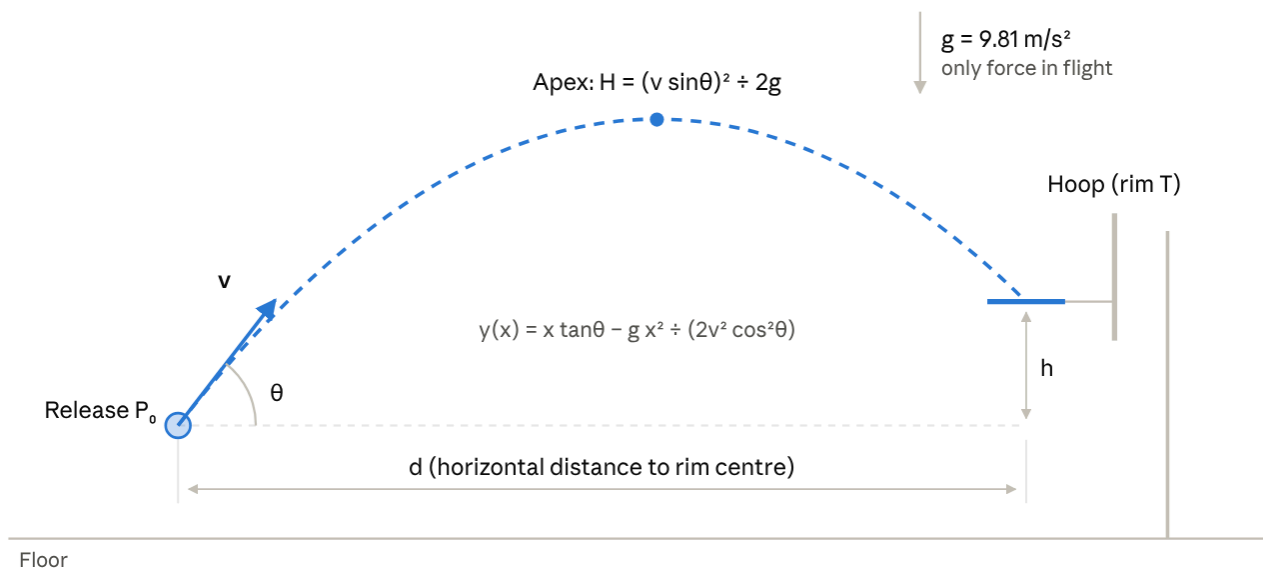
What is expected

- A Unity 6 LTS project (URP) with one playable scene.
- An Android build (.apk / .aab) and an iOS build: landscape, 60 fps on a mid-range phone.
- It is accepted when:
 - ☐ The real ball stays on the preview arc (within 1 cm).
 - ☐ The physics panel reproduces the worked example below.
 - ☐ Aiming with one thumb and shooting with the other works at the same time.
 - ☐ A green arc scores at least 95% of the time.

Physics and calculation

After release, only gravity acts on the ball, so it flies in a parabola set by the launch angle θ and speed v .

The ball follows a parabola from the hand to the rim



shot geometry · release point, launch angle, rim

Path of the ball (draw this as the arc):

$$y(x) = x \tan \theta - \frac{g x^2}{2v^2 \cos^2 \theta}$$

Speed needed to reach the rim (the "ideal speed" on the panel; if $d \times \tan \theta \leq h$, the angle is too low):

$$v = \sqrt{\frac{g d^2}{2 \cos^2 \theta} + (d \tan \theta - h)^2}$$

Other panel values:

$$t_{\text{flight}} = \frac{d}{v \cos \theta} \quad H_{\text{apex}} = \frac{(v \sin \theta)^2}{2g}$$

Worked example (free throw). Release at 2.30 m, rim at 3.05 m, so $h = 0.75$ m. $d = 4.20$ m, and the player chooses $\theta = 52^\circ$.

Value	Result
Speed needed v	7.02 m/s
Horizontal / vertical speed	4.33 / 5.54 m/s
Flight time	0.97 s
Apex	3.86 m above the floor
Entry angle into the rim	42.7°

Value	Result
Angle that needs the least power	50.1° (7.01 m/s)

In Unity

- Throw velocity = flat direction to the hoop $\times v \cos\theta$ + up $\times v \sin\theta$, set on the ball's Rigidbody at the release frame.
- Draw the preview by stepping position and velocity with `Time.fixedDeltaTime`, the same way Unity's physics does. The textbook curve differs from Unity's stepping by about 5 cm, enough to turn a make into a miss.